Chapters 5 & 6
Deep Learning (Ian Goodfellow)
Lecture Notes

Contents

1 CHAPTER 5: MACHINE LEARNING BASICS **4**
1.1 Section 5.1: What is Machine Learning? . . . 4
1.1.1 Understanding the Three Components . . . 4
1.1.2 Common Machine Learning Tasks Explained Simply . . . 4
1.2 Section 5.2: Supervised vs Unsupervised Learning . . . 5
1.2.1 A Story to Understand the Difference . . . 5
1.2.2 Supervised Learning Example: Email Spam Filter . . . 5
1.2.3 Unsupervised Learning Example: Customer Segmentation . . . 5
1.2.4 Why the Distinction Matters . . . 6
1.3 Section 5.3: The Fundamental Challenge - Generalization . . . 6
1.3.1 The Memorization Trap . . . 6
1.3.2 Underfitting - When You Don't Learn Enough . . . 6
1.3.3 Overfitting - When You Learn Too Much (The Wrong Things) . . . 7
1.3.4 The Capacity Sweet Spot . . . 7
1.4 Section 5.4: The No Free Lunch Theorem - Why Context Matters . . . 7
1.4.1 The Universal Tool Myth . . . 7
1.4.2 What the Theorem Actually Says . . . 8
1.4.3 What This Means For You . . . 8
1.5 Section 5.5: Regularization - Making Models Behave . . . 8
1.5.1 Understanding Through Analogies . . . 9
1.5.2 Weight Decay - The Most Common Regularization . . . 9
1.5.3 Understanding Lambda (λ) . . . 9
1.5.4 Different Types of Regularization . . . 9
1.6 Section 5.6: Hyperparameters and Validation - Making Smart Choices . . . 10
1.6.1 Parameters vs Hyperparameters . . . 10
1.6.2 The Solution: Three-Way Split . . . 10
1.6.3 Cross-Validation - When Data is Scarce . . . 10
1.7 Section 5.7: Bias and Variance - Understanding Estimator Quality . . . 11
1.7.1 The Target Practice Analogy . . . 11
1.7.2 Mathematical Understanding . . . 11
1.7.3 The Sample Mean Example . . . 11
1.7.4 The Sample Variance Example . . . 11
1.7.5 The Bias-Variance Trade-off . . . 12
1.8 Section 5.8: Maximum Likelihood Estimation - Finding the Best Parameters . . . 12
1.8.1 Intuitive Understanding . . . 12
1.8.2 Step-by-Step Reasoning . . . 12

1.8.3 Connection to Familiar Concepts . . . 12
1.8.4 Properties That Make MLE Special . . . 13
1.9 Section 5.9: Bayesian Estimation - Incorporating Prior Knowledge . . . 13
1.9.1 The Prior - What We Believe Before Seeing Data . . . 13
1.9.2 Bayes' Rule . . . 13
1.9.3 Real-World Analogy: Medical Diagnosis . . . 13
1.9.4 MAP Estimation - A Practical Compromise . . . 13
1.9.5 MAP = MLE + Regularization . . . 14

2 CHAPTER 6: FEEDFORWARD DEEP NETWORKS 15
2.1 Section 6.1: Building a Neural Network - The Basic Architecture . . . 15
2.1.1 The Layer Cake Architecture . . . 15
2.1.2 A Simple Network: Step by Step . . . 15
2.1.3 What Each Layer Learns . . . 15
2.2 Section 6.2: Activation Functions - Adding Non-Linearity . . . 16
2.2.1 Why We Need Non-Linearity . . . 16
2.2.2 Activation Functions Comparison . . . 16
2.2.3 Softmax - Special for Multi-Class . . . 16
2.3 Section 6.3: Loss Functions - Measuring Error . . . 17
2.3.1 The Maximum Likelihood Principle . . . 17
2.3.2 Loss Functions by Task Type . . . 17
2.3.3 Binary Classification Loss . . . 17
2.3.4 Multi-Class Classification Loss . . . 17
2.3.5 Regression Loss . . . 17
2.4 Section 6.4: Back-Propagation - Learning from Mistakes . . . 18
2.4.1 The Chain Rule Foundation . . . 18
2.4.2 Forward Propagation - Making a Prediction . . . 18
2.4.3 Backward Propagation - Computing Gradients . . . 18
2.4.4 fprop and bprop - The Building Blocks . . . 19
2.5 Section 6.5: The Reparameterization Trick - Differentiable Randomness . . . 19
2.5.1 The Problem with Sampling . . . 19
2.5.2 The Trick . . . 19
2.6 Section 6.6: Universal Approximation - Why Networks Are Powerful . . . 19
2.6.1 The Theorem Explained . . . 20
2.6.2 The Power of Depth . . . 20
2.7 Section 6.7: Feature Learning - Discovering Representations . . . 20
2.7.1 The Feature Engineering Problem . . . 20
2.7.2 Learned Features . . . 21
2.8 Section 6.8: Piecewise Linear Units - Modern Hidden Units . . . 21
2.8.1 The ReLU Family . . . 21
2.8.2 Maxout Units . . . 21
2.8.3 Why Piecewise Linear Works . . . 22
2.9 Section 6.9: Historical Context - How We Got Here . . . 22
2.9.1 The Timeline . . . 22
2.9.2 Key Insights That Changed Everything . . . 22

3 QUICK REFERENCE SUMMARY 23
3.1 Key Concepts from Chapter 5 . . . 23
3.2 Key Concepts from Chapter 6 . . . 23
3.3 Important Formulas to Remember . . . 24

These notes are AI generated and designed to help you understand the fundamental concepts of machine learning and deep neural networks. They focus on:

- **Simple explanations** - No complex language
- **Key points only** - What you really need to understand
- **Practical examples** - Real-world analogies
- **Step-by-step reasoning** - Building understanding gradually

Study Plan

1. Read one section at a time
2. Understand the **KEY IDEA** at the beginning of each section
3. Work through the examples
4. Try to explain the concept to yourself without looking

Please check the textbook for errors (if any) as the notes are generated by AI

1 CHAPTER 5: MACHINE LEARNING BASICS

1.1 Section 5.1: What is Machine Learning?

KEY IDEA

Every machine learning system has three essential components - the task it performs, the data it learns from, and how we measure its success.

1.1.1 Understanding the Three Components

Think of learning to play a new video game:

The Task (T) - What you want to accomplish

- In a game: Defeat the final boss, complete a level, achieve a high score
- In machine learning: Recognize faces, translate languages, predict prices

The Experience (E) - How you learn

- In a game: Playing levels, practicing moves, learning from mistakes
- In machine learning: The dataset - examples shown to the algorithm

The Performance Measure (P) - How you know you're improving

- In a game: Score increasing, levels completed, bosses defeated
- In machine learning: Accuracy, error rate, mean squared error

1.1.2 Common Machine Learning Tasks Explained Simply

Classification Imagine you have a set of boxes labeled "spam" and "not spam." Your job is to put each email into the correct box. That's classification - assigning inputs to categories.

Real example: A self-driving car needs to classify objects on the road - is that a pedestrian, another car, a traffic sign, or a tree? Each object goes into a category.

Regression Instead of categories, you need to predict a number. How much will this house sell for? What will the temperature be tomorrow? How many people will visit this website?

Real example: A weather forecasting system predicts the exact temperature (25.3°C), not just "hot" or "cold."

Transcription Converting something from one form to another. Like taking spoken words and writing them down, or reading text from a photograph.

Real example: When you speak to your phone and it types out your message, that's transcription - speech to text.

Translation A special kind of transcription where you convert between languages.

Real example: Google Translate converting English to Hindi while preserving meaning.

Anomaly Detection Finding things that don't belong - like spotting a counterfeit note among real currency.

Real example: Credit card companies detect fraud when someone makes a purchase very different from your usual pattern.

1.2 Section 5.2: Supervised vs Unsupervised Learning

KEY IDEA

The main difference is whether our data comes with answers (supervised) or without answers (unsupervised).

1.2.1 A Story to Understand the Difference

Imagine you're a child learning about animals:

Supervised Learning - With a Teacher Your parent points to animals and tells you their names:

- "This is a dog" (points to dog)
- "This is a cat" (points to cat)
- "This is a cow" (points to cow)

After many examples, when you see a new animal, you can name it correctly. You learned with labels (the names provided by your parent).

Unsupervised Learning - Without a Teacher You see many animals but no one tells you their names. You start noticing patterns:

- Some animals have fur and bark - you group them together
- Some animals have feathers and fly - you group them together
- Some animals have scales and swim - you group them together

You discovered categories yourself! You didn't learn the names "dog" or "bird," but you learned that different types exist.

1.2.2 Supervised Learning Example: Email Spam Filter

Component	What it is
Input (x)	Email content, sender, subject
Output (y)	"spam" or "not spam" (the label)
Training data	Thousands of emails, each marked as spam or not
What's learned	Patterns that distinguish spam from legitimate email

The algorithm learns: Emails with words like "lottery," "winner," "urgent" are more likely spam. Emails from known contacts are less likely spam.

1.2.3 Unsupervised Learning Example: Customer Segmentation

Component	What it is
Input (x)	Customer purchase history, age, location
Output	No labels provided
Training data	Customer records without any categories
What's learned	Natural groupings of customers

The algorithm might discover:

- Group A: Young customers who buy electronics
- Group B: Middle-aged customers who buy home goods
- Group C: Senior customers who buy health products

1.2.4 Why the Distinction Matters

Aspect	Supervised	Unsupervised
Human effort	High (need to label data)	Low (data is raw)
Cost	Expensive (labeling takes time)	Cheap (data is abundant)
Result	Specific predictions	Discover patterns
Use case	When you know what you want	When exploring unknown data

1.3 Section 5.3: The Fundamental Challenge - Generalization

KEY IDEA

We don't just want machines to memorize - we want them to learn patterns that work on new, unseen data.

1.3.1 The Memorization Trap

Imagine two students preparing for an exam:

Student A memorizes answers to 100 practice questions without understanding. On the exam, only 10 questions are from the practice set. Student A gets those 10 right but fails the other 90.

Student B understands the concepts behind the practice questions. Even though the exam questions are new, Student B can figure out the answers.

- **Student A overfitted** to the practice questions.
- **Student B generalized** to new questions.

1.3.2 Underfitting - When You Don't Learn Enough

Scenario: You're learning to identify cars. You only learn one feature: "cars have wheels."
Result:

- You correctly identify cars (they all have wheels)
- You also identify motorcycles, bicycles, and wheelchairs as cars (too many false positives)
- Your understanding is too simple - you underfitted

In machine learning: The model is too simple to capture the true patterns in data.

1.3.3 Overfitting - When You Learn Too Much (The Wrong Things)

Scenario: You're learning to identify cars. You memorize every detail of 10 specific cars:

- Car 1: Red Toyota with license plate ABC123 and a small scratch on left door
- Car 2: Blue Honda with license plate XYZ789 and a fuzzy dice hanging

Result:

- You perfectly identify those 10 cars
- You fail to identify any other car, even a red Toyota with different license plate
- You learned irrelevant details - you overfitted

In machine learning: The model learns noise and irrelevant patterns from training data.

1.3.4 The Capacity Sweet Spot

Think of model capacity like a suitcase size:

Capacity	What fits	Problem
Too small	Can't pack enough	Underfitting - missing important items
Just right	Everything needed	Perfect generalization
Too large	Pack everything including junk	Overfitting - bringing unnecessary items

1.4 Section 5.4: The No Free Lunch Theorem - Why Context Matters

KEY IDEA

No single algorithm is best for everything. The right choice depends on your specific problem.

1.4.1 The Universal Tool Myth

Imagine a toolbox with only one tool - a hammer. You can use it for:

- Hammering nails (perfect!)
- Cutting wood (terrible)
- Tightening screws (destroys the screw)
- Measuring length (not accurate)

A hammer is excellent for its intended purpose but useless for others.

1.4.2 What the Theorem Actually Says

If you average performance over ALL possible problems, every algorithm performs equally well.

This sounds disappointing, but here's why it's not:

The Hidden Assumption: "ALL possible problems" includes nonsense problems like:

- Predicting tomorrow's weather from yesterday's grocery list
- Classifying images based on random noise
- Finding patterns where none exist

The Reality: We don't care about nonsense problems. We care about real-world problems that have structure:

- Images have objects, colors, shapes
- Language has grammar, meaning, context
- Time series have trends, cycles, patterns

1.4.3 What This Means For You

When choosing an algorithm:

1. **Understand your data** - What patterns might exist?
2. **Use prior knowledge** - What do you already know about the problem?
3. **Match algorithm to task** - Different algorithms have different strengths

Example: Choosing an Algorithm for Different Tasks

Task	Good Choice	Why
Face recognition	Deep neural network	Can learn complex visual patterns
Predicting stock prices	Time series models	Captures temporal dependencies
Grouping customers	K-means clustering	Finds natural groupings
Spam detection	Naive Bayes	Works well with text features

1.5 Section 5.5: Regularization - Making Models Behave

KEY IDEA

Regularization is like giving rules to a creative student - guiding them toward better solutions without stifling their learning.

1.5.1 Understanding Through Analogies

Analogy 1: Writing an Essay

- Without rules: Student writes 100 pages, includes everything, loses focus
- With rules: "Maximum 10 pages, must have thesis statement, cite sources"

The rules don't tell the student what to write, but guide toward better essays.

Analogy 2: Cooking

- Without rules: Chef uses every ingredient in kitchen - chaotic dish
- With rules: "Use no more than 5 ingredients" - focused, balanced dish

1.5.2 Weight Decay - The Most Common Regularization

The Basic Idea:

- We want the model to fit the data
- BUT we also want the weights (parameters) to stay small
- Small weights = simpler model = less likely to overfit

The Math (Don't Panic!):

$$J(\mathbf{w}) = \text{Error on training data} + \lambda \times \text{Size of weights}$$

Think of it as:

$$\text{Total Cost} = \text{How wrong we are} + \text{How complicated we are}$$

1.5.3 Understanding Lambda (λ)

λ value	Effect	Result
$\lambda = 0$	No penalty for complexity	Model can be as complicated as it wants - risk overfitting
Small λ	Mild preference for simplicity	Good balance if chosen well
Large λ	Strong preference for simplicity	Model forced to be too simple - risk underfitting

1.5.4 Different Types of Regularization

Type	What it does	Analogy
L2 (weight decay)	Penalizes large weights	"Don't rely too much on any single feature"
L1	Forces some weights to zero	"Choose only the most important features"
Dropout	Randomly ignores neurons during training	"Learn redundantly - don't depend on others"
Early stopping	Stop training before overfitting	"Stop studying when you know enough"

1.6 Section 5.6: Hyperparameters and Validation - Making Smart Choices

KEY IDEA

Hyperparameters are the settings we choose before learning begins. We need a separate validation set to choose them wisely.

1.6.1 Parameters vs Hyperparameters

Think of baking a cake:

Parameters (learned during training):

- How much flour? How many eggs? How much sugar?
- These come from the recipe - learned from experience

Hyperparameters (chosen before training):

- What temperature to bake at? How long to bake?
- These are settings you choose based on your oven and cake type

1.6.2 The Solution: Three-Way Split

All Data is split into:

- **Training Set (60%)** - Learn parameters
- **Validation Set (20%)** - Choose hyperparameters
- **Test Set (20%)** - Final evaluation ONLY

The Process:

1. Use training set to learn weights (parameters)
2. Try different hyperparameters, see what works best on validation set
3. Pick the hyperparameters with best validation performance
4. ONLY ONCE, evaluate on test set to get true performance

1.6.3 Cross-Validation - When Data is Scarce

The Problem: With small datasets, setting aside 20% for validation might leave too little for training.

The Solution: k-fold cross-validation (with k=5):

- Round 1: [TEST][TRAIN][TRAIN][TRAIN][TRAIN]
- Round 2: [TRAIN][TEST][TRAIN][TRAIN][TRAIN]
- Round 3: [TRAIN][TRAIN][TEST][TRAIN][TRAIN]
- Round 4: [TRAIN][TRAIN][TRAIN][TEST][TRAIN]
- Round 5: [TRAIN][TRAIN][TRAIN][TRAIN][TEST]

Every data point gets to be in the test set exactly once. Average the results for reliable estimate.

1.7 Section 5.7: Bias and Variance - Understanding Estimator Quality

KEY IDEA

Every estimate has two types of error - systematic wrongness (bias) and inconsistency (variance).

1.7.1 The Target Practice Analogy

Imagine shooting arrows at a target:

Low Bias, Low Variance (The Ideal) All arrows cluster tightly at the bullseye. Perfect!

Low Bias, High Variance Arrows spread widely but center around bullseye. Average is correct, but individual shots are unpredictable.

High Bias, Low Variance Arrows cluster tightly but away from bullseye. Consistent but systematically wrong.

High Bias, High Variance Arrows spread widely and away from bullseye. Worst of both worlds - wrong on average and unpredictable.

1.7.2 Mathematical Understanding

Bias = How far off are we on average?

- If you repeatedly sample and estimate, what's the average error?
- Low bias = centered around truth
- High bias = systematically off

Variance = How much do estimates vary?

- If you repeatedly sample, how spread out are the estimates?
- Low variance = consistent, reliable
- High variance = unpredictable, unstable

1.7.3 The Sample Mean Example

Scenario: You want to know the average height of students in your college (true mean μ).

Method: Sample 10 students, calculate their average height (sample mean $\hat{\mu}$).

Is $\hat{\mu}$ biased? No! If you repeated this many times with different samples of 10, the average of all these sample means would equal the true population mean. The sample mean is unbiased.

1.7.4 The Sample Variance Example

Scenario: You want to know how much heights vary (true variance σ^2).

Naive method: Calculate variance using the formula with division by m (number of samples).

$$\hat{\sigma}^2 = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \hat{\mu})^2$$

The problem: Because you used the sample mean (which itself varies) instead of the true mean, your estimate is slightly too small on average. It's biased downward.

The fix: Divide by $m - 1$ instead of m:

$$\tilde{\sigma}^2 = \frac{1}{m-1} \sum_{i=1}^{m} (x^{(i)} - \hat{\mu})^2$$

1.7.5 The Bias-Variance Trade-off

The Math:

$$\text{MSE} = \text{Bias}^2 + \text{Variance}$$

Total error = systematic error^2 + unpredictable error

1.8 Section 5.8: Maximum Likelihood Estimation - Finding the Best Parameters

KEY IDEA

Choose parameters that make the observed data most probable.

1.8.1 Intuitive Understanding

Imagine you find a bag of marbles. You draw 10 marbles:

- 7 red, 3 blue

What's your best guess for the true proportion of red marbles in the bag?

Maximum likelihood answer: 70% red

Why? Because if the true proportion were 70%, the probability of observing exactly 7 red out of 10 is maximized.

1.8.2 Step-by-Step Reasoning

Step 1: Define the likelihood The likelihood is: "Given parameters θ, how probable is the data we observed?"

Step 2: Write it mathematically

$$L(\theta) = P(\text{data}|\theta) = \prod_{i=1}^{m} P(x^{(i)}|\theta)$$

Step 3: Take log (makes math easier)

$$\log L(\theta) = \sum_{i=1}^{m} \log P(x^{(i)}|\theta)$$

Step 4: Maximize Find θ that makes this sum largest

1.8.3 Connection to Familiar Concepts

- **For regression with Gaussian errors:** Maximum likelihood = minimizing mean squared error
- **For binary classification with Bernoulli:** Maximum likelihood = minimizing cross-entropy

1.8.4 Properties That Make MLE Special

Property	What it means
Consistency	As data increases, estimate approaches true value
Efficiency	Lowest possible variance among consistent estimators
Invariance	If $\hat{\theta}$ is MLE for θ, then $f(\hat{\theta})$ is MLE for $f(\theta)$

1.9 Section 5.9: Bayesian Estimation - Incorporating Prior Knowledge

KEY IDEA

Instead of a single best estimate, maintain a distribution of possibilities that updates with evidence.

1.9.1 The Prior - What We Believe Before Seeing Data

Imagine you're estimating the fairness of a coin:

Skeptic's prior: "I believe any fairness is equally possible" (uniform from 0 to 1)

Optimist's prior: "I believe most coins are roughly fair" (bell curve centered at 0.5)

Pessimist's prior: "I believe coins might be biased" (bimodal with peaks near 0 and 1)

1.9.2 Bayes' Rule

$$P(\theta|\text{data}) \propto P(\text{data}|\theta) \times P(\theta)$$

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

1.9.3 Real-World Analogy: Medical Diagnosis

- **Prior:** Only 1% of population has disease X
- **Likelihood:** Test is 95% accurate (95% sensitivity, 95% specificity)
- **Posterior:** You test positive. What's the probability you have the disease?

Even with positive test, probability is not 95% because the disease is rare. Bayesian reasoning correctly accounts for this.

1.9.4 MAP Estimation - A Practical Compromise

Maximum A Posteriori (MAP) gives us a single best estimate:

$$\theta_{\text{MAP}} = \arg\max_{\theta}[\log P(\text{data}|\theta) + \log P(\theta)]$$

Compare with MLE:

$$\theta_{\text{MLE}} = \arg\max_{\theta} \log P(\text{data}|\theta)$$

The only difference is the $\log P(\theta)$ term - the prior!

1.9.5 MAP = MLE + Regularization

Key insight: When prior is Gaussian, MAP adds a penalty for large weights:

$$\log P(\theta) \propto -\lambda ||\theta||^2$$

This is exactly weight decay regularization!

2 CHAPTER 6: FEEDFORWARD DEEP NETWORKS

2.1 Section 6.1: Building a Neural Network - The Basic Architecture

KEY IDEA

Neural networks learn by transforming data through multiple layers, each extracting more useful features.

2.1.1 The Layer Cake Architecture

```
Input Layer (raw ingredients)
    ↓
Hidden Layer 1 (mix ingredients)
    ↓
Hidden Layer 2 (bake the mixture)
    ↓
Hidden Layer 3 (add frosting)
    ↓
Output Layer (final decorated cake)
```

Each layer transforms what came before, building complexity step by step.

2.1.2 A Simple Network: Step by Step

Step 1: Take input

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]$$

Step 2: First transformation

$$\mathbf{a}^{(1)} = \mathbf{b}^{(1)} + \mathbf{W}^{(1)}\mathbf{x}$$

Step 3: Add non-linearity

$$\mathbf{h}^{(1)} = \text{sigmoid}(\mathbf{a}^{(1)})$$

Step 4: Repeat for more layers

$$\mathbf{a}^{(2)} = \mathbf{b}^{(2)} + \mathbf{W}^{(2)}\mathbf{h}^{(1)}$$

$$\mathbf{h}^{(2)} = \text{sigmoid}(\mathbf{a}^{(2)})$$

Step 5: Final output

$$\hat{\mathbf{y}} = \mathbf{b}^{(3)} + \mathbf{W}^{(3)}\mathbf{h}^{(2)}$$

2.1.3 What Each Layer Learns

In an image recognition network:

- **Layer 1:** Detects edges (horizontal, vertical, diagonal)
- **Layer 2:** Combines edges into simple shapes (circles, corners)
- **Layer 3:** Combines shapes into parts (eyes, wheels, windows)
- **Layer 4:** Combines parts into objects (faces, cars, houses)

2.2 Section 6.2: Activation Functions - Adding Non-Linearity

KEY IDEA

Activation functions introduce non-linearity, allowing networks to learn complex patterns.

2.2.1 Why We Need Non-Linearity

Without non-linearity:

$$\begin{aligned} \text{Layer 1: } & \mathbf{h}_1 = \mathbf{W}_1\mathbf{x} \\ \text{Layer 2: } & \mathbf{h}_2 = \mathbf{W}_2\mathbf{h}_1 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} \end{aligned}$$

This is just one linear transformation! Two layers = one layer

With non-linearity:

$$\begin{aligned} \text{Layer 1: } & \mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x}) \\ \text{Layer 2: } & \mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1) \end{aligned}$$

Now each layer can learn different things

2.2.2 Activation Functions Comparison

Activation	Formula	Properties
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	Output in (0,1), gradients vanish for extreme inputs
Tanh	$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$	Output in (-1,1), zero-centered, gradients still vanish
ReLU	$\max(0, a)$	No vanishing gradient for positive inputs, computationally cheap
Leaky ReLU	$\max(0, a) + 0.01 \min(0, a)$	Fixes dead ReLU problem
PReLU	$\max(0, a) + \alpha \min(0, a)$	Slope α learned during training

2.2.3 Softmax - Special for Multi-Class

$$\text{softmax}(a)_i = \frac{e^{a_i}}{\sum_j e^{a_j}}$$

Properties:

- All outputs positive
- All outputs sum to 1
- Can interpret as probabilities

Example: For 3 classes with scores [2, 1, 0.1]: Softmax outputs [0.65, 0.24, 0.11] (class 1 most likely)

2.3 Section 6.3: Loss Functions - Measuring Error

KEY IDEA

The loss function tells the network how wrong it is, guiding learning.

2.3.1 The Maximum Likelihood Principle

Instead of inventing loss functions, use maximum likelihood:

$$L(\mathbf{x}, \mathbf{y}) = -\log P(\mathbf{y}|\mathbf{x})$$

2.3.2 Loss Functions by Task Type

Task	Activation	Loss Function
Binary classification	Sigmoid	Binary cross-entropy
Multi-class classification	Softmax	Categorical cross-entropy
Regression	Linear	Mean squared error (MSE)

2.3.3 Binary Classification Loss

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

- If true $y = 1$: $L = -\log(p)$ (want p close to 1)
- If true $y = 0$: $L = -\log(1 - p)$ (want p close to 0)

2.3.4 Multi-Class Classification Loss

$$L = -\log(p_{\text{correct}})$$

Gradient:

$$\frac{\partial L}{\partial a_k} = p_k - \mathbf{1}_{k=\text{correct}}$$

- Correct class: gradient negative (push up)
- Wrong classes: gradient positive (push down)

2.3.5 Regression Loss

For Gaussian distribution with mean $\mu = f(x)$:

$$L = \frac{1}{2\sigma^2}(y - \mu)^2 + \text{constant}$$

With fixed σ, this is just MSE!

2.4 Section 6.4: Back-Propagation - Learning from Mistakes

KEY IDEA

Back-propagation efficiently computes how each weight contributed to the error, using the chain rule and dynamic programming.

2.4.1 The Chain Rule Foundation

Simple case: z depends on y, y depends on x

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x}$$

Multiple paths:

$$\frac{\partial z}{\partial x} = \sum_{\text{paths}} \frac{\partial z}{\partial \text{path}_i}$$

2.4.2 Forward Propagation - Making a Prediction

```
Input x
    ↓
Layer 1: a¹ = b¹ + W¹x
         h¹ = (a¹)
    ↓
Layer 2: a² = b² + W²h¹
         h² = (a²)
    ↓
Output: ŷ = b³ + W³h²
    ↓
Loss: J = L(ŷ, y) + regularization
```

2.4.3 Backward Propagation - Computing Gradients

Step 1: Start at the loss

$$\mathbf{g} = \nabla_{\hat{\mathbf{y}}} J$$

Step 2: For layer k from last to first:

1. Gradient through activation:

$$\mathbf{g} = \nabla_{\mathbf{a}^{(k)}} J = \mathbf{g} \odot \sigma'(\mathbf{a}^{(k)})$$

2. Gradients for parameters:

$$\nabla_{\mathbf{b}^{(k)}} J = \mathbf{g}$$
$$\nabla_{\mathbf{W}^{(k)}} J = \mathbf{g}\mathbf{h}^{(k-1)\top}$$

3. Propagate to previous layer:

$$\mathbf{g} = \nabla_{\mathbf{h}^{(k-1)}} J = \mathbf{W}^{(k)\top}\mathbf{g}$$

2.4.4 fprop and bprop - The Building Blocks

fprop (forward):
- Takes inputs
- Computes output
- Stores anything needed for backward

bprop (backward):
- Takes gradient with respect to output
- Computes gradient with respect to inputs
- Uses stored values from fprop

2.5 Section 6.5: The Reparameterization Trick - Differentiable Randomness

KEY IDEA

To back-propagate through random operations, we separate the randomness from the learnable parameters.

2.5.1 The Problem with Sampling

Consider sampling from a Gaussian:

$$y \sim \mathcal{N}(\mu, \sigma^2)$$

We want gradients with respect to μ and σ. But sampling is a random process - how do we differentiate through it?

2.5.2 The Trick

Rewrite the sampling as:

$$y = \mu + \sigma \times \eta, \quad \eta \sim \mathcal{N}(0, 1)$$

Now:

- μ and σ are deterministic
- η provides randomness from fixed distribution
- The whole expression is differentiable!

Before: [Random Generator] $\rightarrow$ CAN'T COMPUTE GRADIENTS!
After: $y = \mu + \sigma \times \eta \rightarrow$ CAN COMPUTE $\partial y/\partial \mu$ and $\partial y/\partial \sigma$!

2.6 Section 6.6: Universal Approximation - Why Networks Are Powerful

KEY IDEA

Neural networks can approximate any function, but depth makes approximation efficient.

2.6.1 The Theorem Explained

Any continuous function on a bounded domain can be approximated arbitrarily well by a neural network with one hidden layer, given enough hidden units.

What this means:

- Need to learn a function? A neural network can do it
- No inherent limitation on what can be represented
- Given enough neurons, you can always get closer to the true function

2.6.2 The Power of Depth

Example: The Parity Function

Computing whether sum of n bits is even or odd:

- Shallow network: needs 2^n hidden units
- Deep network: can do with $O(n)$ units by building hierarchically

Why depth helps:

- Each layer builds on previous
- Complex functions decomposed into simpler ones
- Features reused across the network

2.7 Section 6.7: Feature Learning - Discovering Representations

KEY IDEA

Instead of manually designing features, let the network learn them.

2.7.1 The Feature Engineering Problem

Traditional machine learning:

1. Human expert examines problem
2. Designs features they think are important
3. Model uses these fixed features

Problems:

- Time-consuming
- Requires domain expertise
- May miss important patterns

2.7.2 Learned Features

Deep learning approach:

1. Give raw data to network
2. Network discovers useful features automatically
3. Features build hierarchically

Example: Face Recognition

Layer	What it learns
Layer 1	Edge detectors (horizontal, vertical, diagonal)
Layer 2	Simple shapes (circles, corners)
Layer 3	Parts (eyes, nose, mouth)
Layer 4	Complete face patterns

2.8 Section 6.8: Piecewise Linear Units - Modern Hidden Units

KEY IDEA

Simple piecewise linear functions have become the default choice for hidden layers due to their excellent optimization properties.

2.8.1 The ReLU Family

Unit	Formula	Key Property
ReLU	$\max(0, a)$	Gradient 1 for active units, can die
Leaky ReLU	$\max(0, a) + 0.01 \min(0, a)$	Small slope when negative, never dies
PReLU	$\max(0, a) + \alpha \min(0, a)$	Slope α learned during training

2.8.2 Maxout Units

$$f_i(x) = \max_{j \in G_i} (w_{ij}^\top x + b_{ij})$$

Properties:

- Can approximate any convex function
- Multiple pieces with independent weights
- Never has zero gradient

2.8.3 Why Piecewise Linear Works

1. **No vanishing gradients** - slopes constant in each region
2. **Sparse activation** - many units output 0
3. **Fast computation** - just comparisons
4. **Good gradient flow** - information propagates easily

2.9 Section 6.9: Historical Context - How We Got Here

KEY IDEA

Today's deep learning builds on decades of research, with recent breakthroughs in optimization and hardware.

2.9.1 The Timeline

Period	Development
1940s-1960s	First neural network models, Perceptron invented
1969-1980s	AI Winter - funding dried up, research continued quietly
1986	Back-propagation popularized (Rumelhart, Hinton, Williams)
1990s-2000s	SVMs became popular, neural networks struggled
2006	Deep learning revival with pre-training techniques
2012-Present	Deep learning dominance (AlexNet, GPUs, breakthroughs)

2.9.2 Key Insights That Changed Everything

1. **Better activation functions** - From sigmoid $\rightarrow$ ReLU: Solved vanishing gradient
2. **More data** - ImageNet, web-scale datasets enabled learning
3. **Faster hardware** - GPUs made training practical
4. **Better initialization** - Careful weight starting points help optimization
5. **Regularization advances** - Dropout, batch norm prevent overfitting

3 QUICK REFERENCE SUMMARY

3.1 Key Concepts from Chapter 5

Concept	Simple Definition
Task (T)	What we want the computer to do
Experience (E)	Data the computer learns from
Performance (P)	How we measure success
Supervised learning	Learning with labeled data
Unsupervised learning	Finding patterns without labels
Underfitting	Model too simple - high error everywhere
Overfitting	Model too complex - memorizes training data
Regularization	Any method to reduce overfitting
Hyperparameters	Settings chosen before training
Validation set	Data to tune hyperparameters
Bias	Systematic error - consistently wrong
Variance	Inconsistency - unpredictable predictions
MLE	Parameters that make observed data most likely
MAP	MLE + Prior knowledge (regularization)

3.2 Key Concepts from Chapter 6

Concept	Simple Definition
Feedforward network	Information flows one direction (input $\rightarrow$ output)
Hidden layer	Layer between input and output that learns features
Activation function	Adds non-linearity to network
ReLU	$\max(0, a)$ - popular non-saturating activation
Softmax	Makes outputs sum to 1 (probabilities)
Loss function	Measures how wrong predictions are
Back-propagation	Efficient way to compute gradients
Chain rule	$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x}$
fprop	Forward computation pass
bprop	Backward gradient computation pass
Reparameterization trick	Make sampling differentiable
Universal approximation	Networks can learn any function (given enough units)

Concept	Simple Definition
Feature learning	Network discovers useful features automatically

3.3 Important Formulas to Remember

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \quad \text{(Linear regression)} \tag{1}$$

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 \tag{2}$$

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta \tag{3}$$

$$\text{Var}(\hat{\theta}) = \mathbb{E}[\hat{\theta}^2] - \mathbb{E}[\hat{\theta}]^2 \tag{4}$$

$$\text{MSE} = \text{Bias}^2 + \text{Var} \tag{5}$$

$$\theta_{\text{ML}} = \arg\max_{\theta} \sum_{i=1}^{m} \log p(x^{(i)}|\theta) \tag{6}$$

$$\theta_{\text{MAP}} = \arg\max_{\theta} [\sum_{i=1}^{m} \log p(x^{(i)}|\theta) + \log p(\theta)] \tag{7}$$

$$\sigma(a) = \frac{1}{1+e^{-a}} \tag{8}$$

$$\text{ReLU}(a) = \max(0, a) \tag{9}$$

$$\text{softmax}(a)_i = \frac{e^{a_i}}{\sum_j e^{a_j}} \tag{10}$$

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x} \tag{11}$$

$$y = \mu + \sigma\eta, \quad \eta \sim \mathcal{N}(0,1) \tag{12}$$

Remember: Understanding these concepts is more important than memorizing them. Try to explain each idea to someone else - if you can teach it, you know it!